

**ASIA PACIFIC UNIVERSITY OF TECHNOLOGY & INNOVATION****CT013-3-3-CSS INDIVIDUAL ASSIGNMENT IMPLEMENTATION REPORT**

TITLE	ATTENDANCE MANAGEMENT SYSTEM
NAME & STUDENT ID	NOOR UR RASHID (TP066720)
INTAKE	APD4F2601CE
LECTURER	DR. KAMALAKANNAN MACHAP
Date	27/03/2026

Table of Contents

List of Figures	3
1.0 Introduction.....	4
2.0 Functions and Importance of Security Features.....	5
2.1 Secure Password Storage Using bcrypt	5
2.2 Login Attempt Limitation to Prevent Brute Force Attacks	6
2.3 CAPTCHA Verification Attack to Prevent Automated Attacks	6
2.4 Roal Based Access Control.....	7
2.5 OTP Based Account Recovery System.....	7
2.6 Password Strength Validation	8
3.0 Discussion of Algorithms and Methods.....	9
3.1 Password Hashing Method	9
3.2 Role Based Access Control Method	10
3.3 Login Attempt Limitation Method.....	11
3.4 CAPTCHA Verification Method.....	12
3.5 OTP Generation and Verification Method	13
3.6 Password Strength Validation Method.....	15
3.7 Database Query and Record Checking Method.....	16
3.8 Attendance Percentage Calculation Method	17
4.0 Implementation and Critical Analysis.....	18
4.1 Secure Password Storage Implementation.....	18
4.2 Login Attempt Limitation Implementation.....	19
4.3 CAPTCHA Verification Implementation	20
4.4 OTP-Based Account Recovery Implementation.....	21
4.5 Role-Based Access Control Implementation	22

4.6 Database Validation and Data Integrity Implementation	23
4.7 Attendance Calculation and Data Processing Implementation	25
4.8 Overall System Evaluation	26
5.0 Conclusion	27
6.0 Reference	28

List of Figures

Figure 1 Block Diagram of Password Hashing Method	10
Figure 2 Block Diagram of Role Based Access Control Method	11
Figure 3 Block Diagram of Login Attempt Limitation Method	12
Figure 4 Block Diagram of CAPTCHA Verification Method	13
Figure 5 Block Diagram of OTP Generation and Verification Method	14
Figure 6 Block Diagram of Password Strength Validation Method.....	15
Figure 7 Block Diagram of Database Query and Record Checking Method	16
Figure 8 Block Diagram of Attendance Percentage Calculation Method.....	17
Figure 9 Secure Password Storage Code	19
Figure 10 Login Attempt Limitation Code	20
Figure 11 CAPTCHA Verification Code.....	21
Figure 12 OTP-Based Account Recovery Code	22
Figure 13 Role-Based Access Control Code.....	23
Figure 14 Database Validation and Data Integrity Code	25
Figure 15 Attendance Calculation and Data Processing Code.....	26

1.0 Introduction

This implementation report describes the development and technical implementation of a Web-Based Attendance Management System that will be used to automate the process of recording and managing student attendance in a learning institution. The system has been implemented by using Flask web framework in Python whereby the backend logic, routing, and communication between the user interface and the database are created. The application has incorporated several technologies such as HTML, CSS, Flask-Login, Flask-SQLAlchemy, Flask-WTF, and Flask-Mail which enable the provision of a secure and workable web application to the administrators, teachers and students.

The architecture of the system is that of role-based architecture whereby various permissions and functionalities are granted to various categories of users in the system. Administrators create and manage teacher and student accounts, manage subjects, enroll students into subjects and keep track of attendance statistics. Teachers can manage their subjects, and they can record student attendance to a particular class session. The students are given access to a personal dashboard where they can view the enrolled subjects and check their attendance records. The user accounts, subject information, enrollment records and attendance data are stored in an ordered format in a SQLite relational database that is managed by SQLAlchemy.

In order to secure the systems and guarantee the safety of the user data, a number of security procedures have been applied in the systems development process. Flask-Login takes care of user authentication and password storage is done with the help of the bcrypt hashing algorithm. Other security options have also been included like the limitation of login attempts, CAPTCHA, OTP-unlocking of account, password strength validation and role-based access control. These mechanisms help in providing security against the typical web security threats including brute force attacks, automation of the login process, and unauthorized access to restricted pages.

Generally, the assimilation of contemporary web development and security measures are exhibited in this implementation and are used to create a dependable and safe attendance management system. The technical implementation of the system will be outlined in the subsequent sections of this report such as a system architecture, database models, authentication mechanisms and security features incorporated during implementation.

2.0 Functions and Importance of Security Features

Web-based applications should have security features that protect the user data, system resources and authentication system against unauthorized access and cyberattacks. Some of the security controls, which are employed in the implemented Attendance Management System, have been incorporated in the backend to thwart the typical web security threats, including the brute-force, automated login attempts, weak password exploitation, and unauthorized access to restricted pages. These security systems are deployed on the basis of the latest security practices and libraries by Flask.

The security features fulfill a particular purpose in securing the system without obstructing the usage by legitimacy users. Strong security architecture of the application is achieved through authentication, access control, password protection, CAPTCHA verification, and OTP-based verification. The designation of the functionality and significance of all the security mechanisms in the system is discussed in the following subsections.

2.1 Secure Password Storage Using bcrypt

The algorithm of hashing through bcrypt will ensure that the user passwords are not stored in the database in plain text. Instead, the passwords are hashed with the help of bcrypt and then stored in the database. With bcrypt verification functions, when the user tries to log in the password typed is compared to the stored hashed one. This is done such that the original password is not easy to retrieve in case the database is compromised.

Bcrypt is very useful in enhancing security since salting and numerous hash rounds are used and thus brute-force or dictionary attacks are extremely hard to use. Having access to the passwords database, even in case the attackers retrieve it, the hashed values cannot be easily transformed into the original passwords. This is a popularly suggested strategy to use in storing passwords securely in the web application.

2.2 Login Attempt Limitation to Prevent Brute Force Attacks

The system has been secured against the brute-force attacks by introducing a mechanism that limits the number of attempts that someone can make to use a log in. Brute-force attacks are where attackers constantly try various passwords so as to access the user accounts illegally. The failed attempts variable in the database is used in the implemented system to keep track of the number of failed attempts in the system.

When an end user provides wrong login credentials, failed attempt counter is incremented. After five consecutive unsuccessful attempts, the account is temporarily locked to allow some period of time (five minutes). At this stage, the user will not be allowed to log in with the correct password. There is a countdown timer to indicate to the user how soon the account is available again.

The mechanism can work well in preventing the automated password guessing attacks and ensuring that an attacker cannot quickly enter a number of different login combinations.

2.3 CAPTCHA Verification Attack to Prevent Automated Attacks

CAPTCHA check is introduced to avoid bots or malicious scripts used to perform automated login attacks. It is possible to transmit thousands of login requests to the server within a few minutes as automated attacks attempt to guess passwords or use authentication vulnerabilities. CAPTCHA makes sure that it is not automated Logging in but done by human beings.

The CAPTCHA verification should be enabled in the implemented system when a failure in the attempt to log in happens three times. There is a dynamically generated image with randomly generated characters on the log-in that the user must enter the information displayed correctly before he/she can make another attempt of logging-in. The value of CAPTCHA is saved in the temporary version of the session and is checked with the user input.

Such a system is useful in minimizing the threat of automated attacks since bots do not usually perceive the CAPTCHA images correctly.

2.4 Roal Based Access Control

Implementation of Role-Based Access Control (RBAC) is to make sure that authorized users are only able to see certain pages or system features. The Attendance Management System defines three user roles, including administrator, teacher, and student. Every role is granted certain permissions that define which pages and features within the system could be accessed.

The system identifies the role of the logged in user before a user gains access to a page that is secured by the current user.role condition. In case the user lacks the necessary role the system takes the user to the login page and blocks any unauthorized access.

As an example, administrative tasks like adding user, subject management, and system wide attendance statistics are limited to administrators. Teachers are able to record attendance on subjects they are assigned and the students are only able to check their own attendance history.

2.5 OTP Based Account Recovery System

There is also an OTP (One-Time Password) verification system that is used as a safe way of unlocking user accounts when they are locked as a result of frequent failed log-ins. On locking an account, the user is presented with the choice of confirming who he is by entering an OTP which is sent to the email address he has registered.

OTP is generated randomly and held temporarily in the session along with an expiration period. The OTP is not permanent and has a short lifespan (one minute) when it goes out of date automatically. To unlock the account, the user has to input the right OTP before the expiration period. After a successful verification, the counter of the attempts to log in and the status of locking an account is returned to zero.

This system makes sure that the account is only unlocked by the authentic user of the registered email address hence avoiding the unauthorized recovery efforts.

2.6 Password Strength Validation

The password strength verification has also been employed to make sure that the user is entering strong and secure passwords whenever setting or updating their credentials. One of the most prevalent reasons of security breach is weak passwords that are easily guessed by a dictionary attack or brute-force attack.

Password validation rules in the system place a minimum requirement of a password length which consists of eight characters comprising a mix of uppercase letters, lower case letters, numbers and special characters. These conditions make password complexity very high and it lowers the chances of password guessing attacks.

Authentication of the password is done on the client and server side and the database only accepts the passwords that are validated.

3.0 Discussion of Algorithms and Methods

There were various algorithms and techniques employed during the implementation of the Attendance Management System to ensure that the system is running in the right way, safely and efficiently. These were used in authentication of users, protection of passwords, verification of roles, processing of attendance, record management and recovery of accounts. The system was primarily executed through structured backend techniques, conditional logic, database queries, hash functions and validation processes instead of taking advanced machine learning algorithms. They were chosen according to their appropriateness to the construction of the secure web-based application with the help of Flask.

3.1 Password Hashing Method

User passwords were stored in the database in the form of password hashes that were obtained by hashing user passwords. The bcrypt hashing algorithm was used in the system to encrypt passwords into hash values. The technique was used in the case of temporary passwords to be given to teachers and students and also when users altered their passwords upon first login. In authentication, the password typed in was run through the verification of the hashed password registered in the system using the bcrypt verification rather than being directly compared with the actual password.

The technique is significant because the storage of plain-text passwords has been avoided. As a result, even in the situation when the database was used by an intruder, the original passwords would not be revealed. Bcrypt was thus chosen as an appropriate way of secure credential protection in the system.

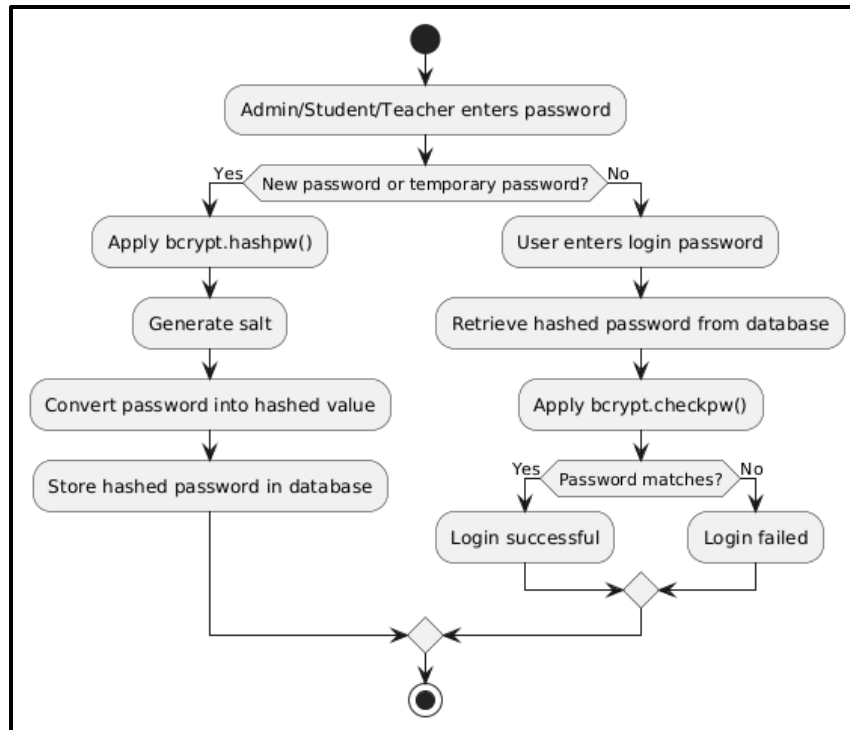


Figure 1 Block Diagram of Password Hashing Method

3.2 Role Based Access Control Method

An access control strategy was adopted as a role-based access control to control access permission of various users within the system. Users in the project were divided into three roles, including the role of an admin, teacher and student. The backend routes employed conditional checking to confirm the role of the user who was logged in prior to being allowed to access certain pages. An example of this is that only the administrators could access user management and subject creation pages, only the teachers could record attendance, and only the students could view their own attendance information.

This was done to make sure that the users could only access functions that were applicable to them. Using this access control technique, sensitive pages, and system operations were not accessed by unauthorized people.

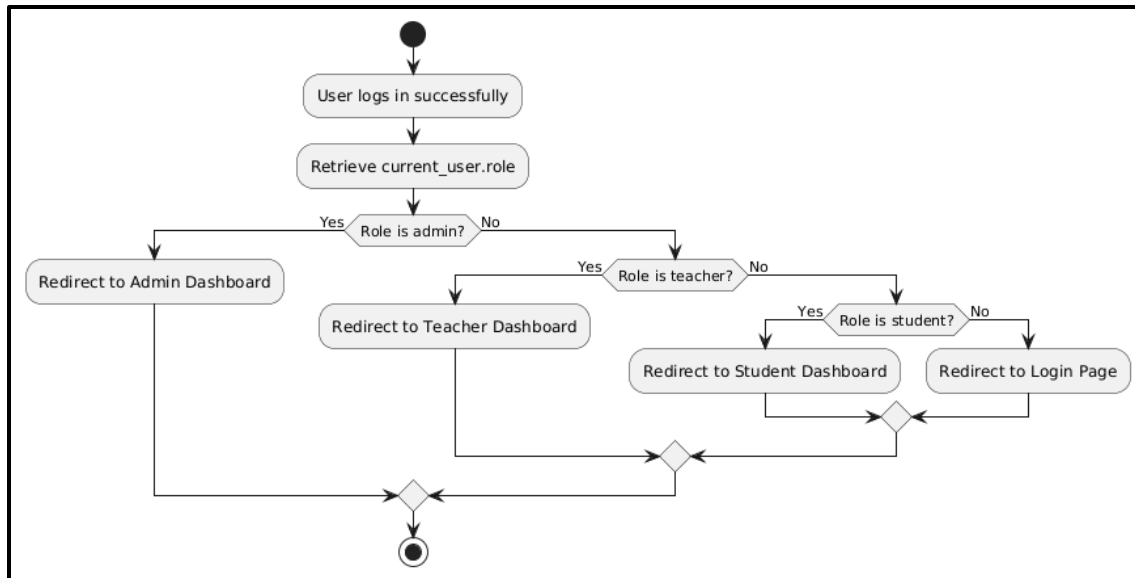


Figure 2 Block Diagram of Role Based Access Control Method

3.3 Login Attempt Limitation Method

To minimize the chances of repeated unauthorized logins, a method of limiting a login attempt was implemented. During the process of login in, the failed_attempts field was used to store the number of failed attempts in the database. The counter was added every time a wrong password was typed. After five unsuccessful attempts, the account was temporarily locked with the help of the lock_until field and no user could log in until the expiration of the lock period or OTP verification.

This was done as a defines mechanism against brute force attacks. The system was resistant to password guessing attacks and automated login abuse by reducing the number of repetitions.

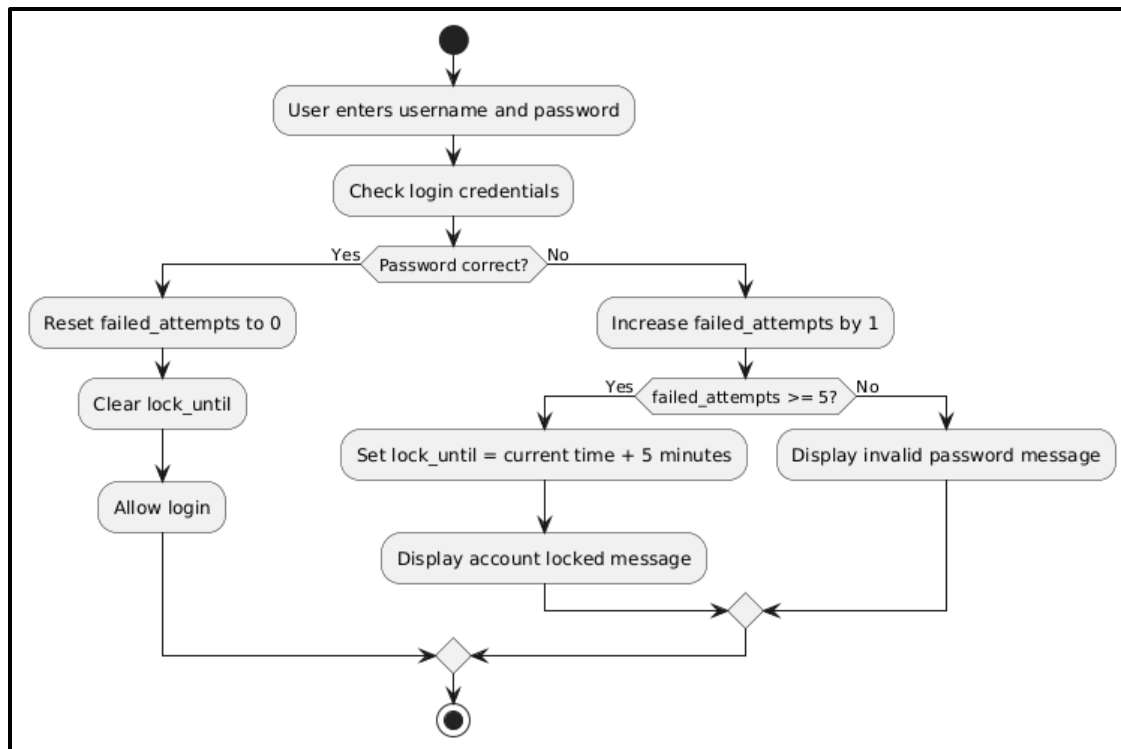


Figure 3 Block Diagram of Login Attempt Limitation Method

3.4 CAPTCHA Verification Method

After several consecutive failed attempts at logging in, there was a CAPTCHA verification procedure. Once the number of failed logins was three, CAPTCHA input was mandatory and the next attempt of logging in could be continued. CAPTCHA text was dynamically obtained with the help of the `generate_captcha()` function and placed in a session. The user entered value was then matched against the generated CAPTCHA value which was stored in the session.

This was done in order to differentiate between human users and automated bots. CAPTCHA validation was added since automated scripts could attempt to log-in into the page at high speed multiple times, thereby decreasing the number of automated attacks on the login page.

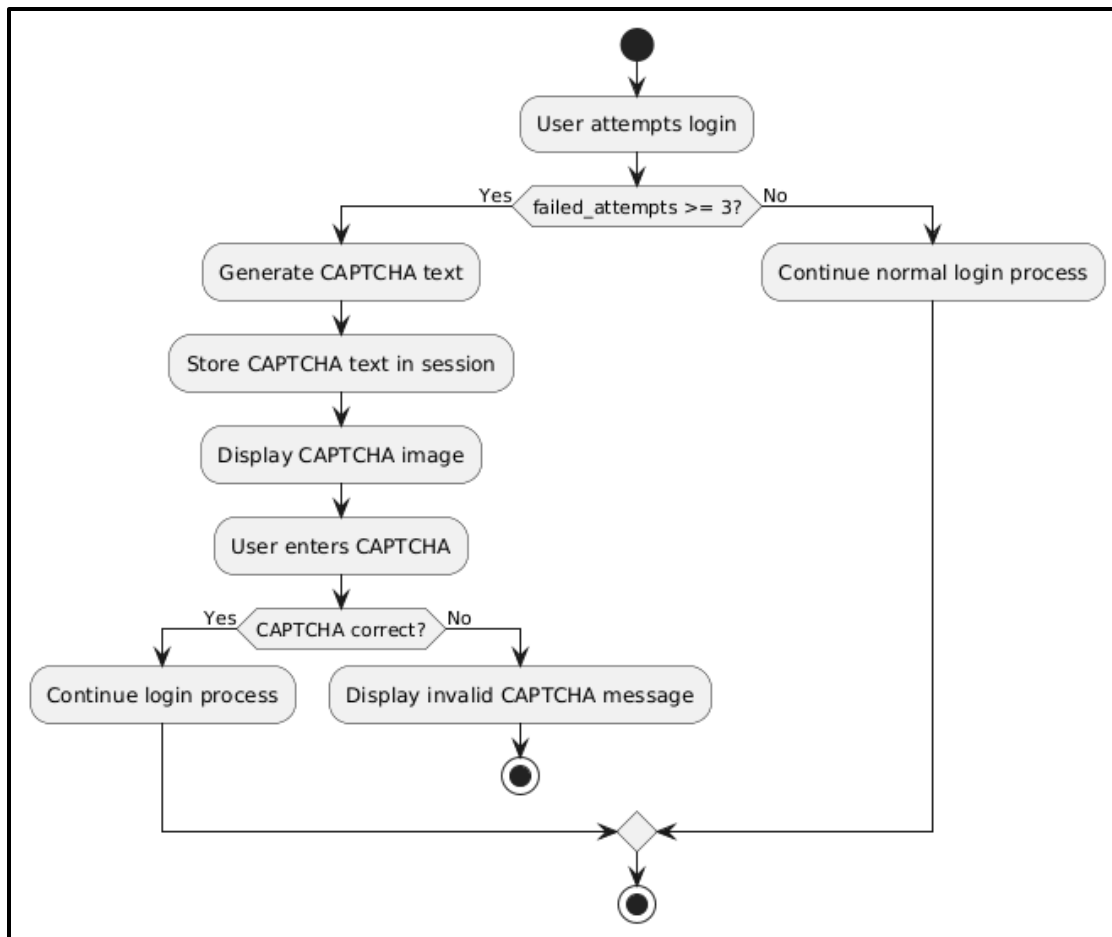


Figure 4 Block Diagram of CAPTCHA Verification Method

3.5 OTP Generation and Verification Method

An OTP-based account recovery was also introduced in case an account was locked. An OTP with six digits was created by a random number with the function `random.randint()` and mailed to the registered email address of the user using Flask-Mail. The OTP and its time of expiration was temporarily stored in the session. In verification the OTP typed in by the user was matched against the stored OTP and the present time was matched to the expiration time.

This was done to give a safe way of recovery without manual administrator intervention. After the OTP was successfully checked, the unsuccessful attempts at logging in were cleared and the account lock was eliminated. Such a technique enhanced the usability and account security.

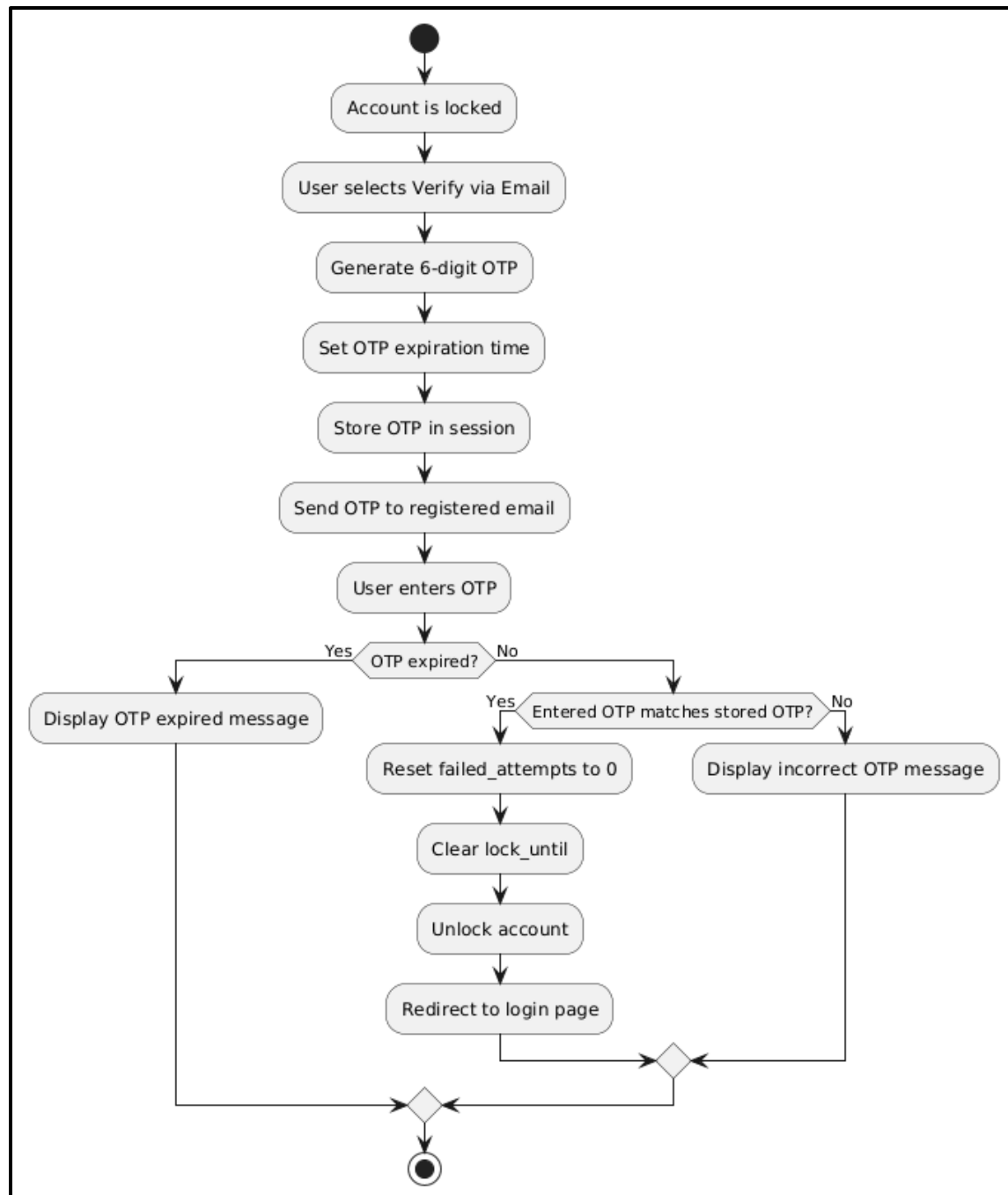


Figure 5 Block Diagram of OTP Generation and Verification Method

3.6 Password Strength Validation Method

Pattern matching was adopted to give a password validation method to ensure that strong passwords were generated. The verification of the presence of upper case letters, lower case letters, numbers, special characters and a minimum length of eight characters present in the password was done by the use of regular expressions in the project. This authentication was used in the `is_password_valid()` function and was used where a user changed a temporary password.

This was done to minimize the possibility of having weak passwords chosen by the user. The complexity of passwords rules were implemented, which enhanced the security of each account of users and minimized the possibility of successful password guessing.

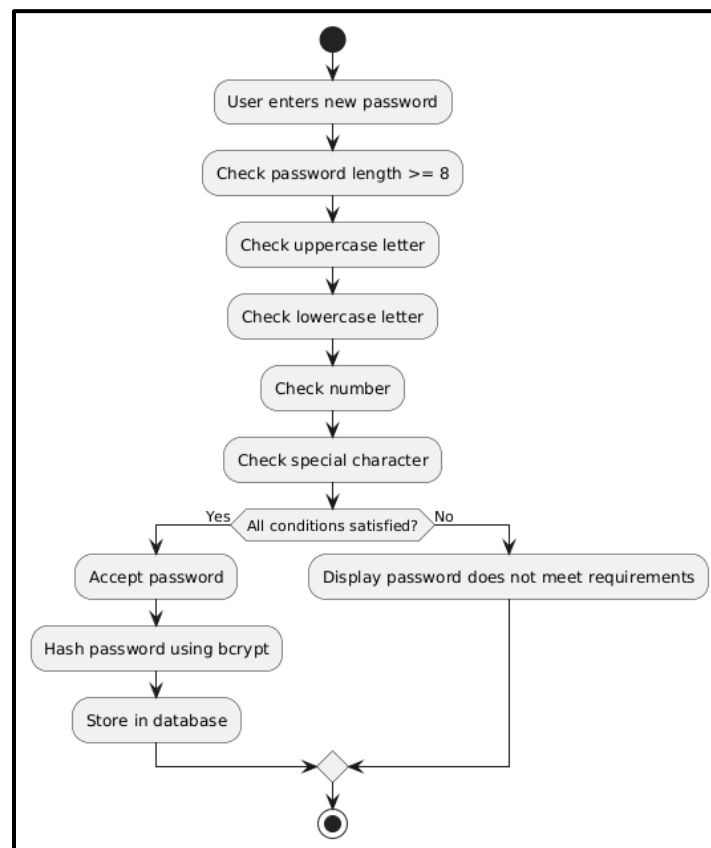


Figure 6 Block Diagram of Password Strength Validation Method

3.7 Database Query and Record Checking Method

The system utilized a database query and record scanning approach in the management of users, subjects, enrollments, and the attendance records. Structured queries were done through SQLAlchemy ORM which helped with retrieving a user by their username, determining when two or more usernames or emails were the same, when one, not yet registered in a subject, was already registered in the same subject, and whether a student was already marked as already attended on the relevant day.

This technique was particularly critical in marking of attendance and creation of accounts. An example of this was the outcome of a duplicate attend record, which was avoided by examining whether another record of the same student, subject and date had previously been entered before another attendance record was added. In the same way, the same usernames, emails and subject names were not allowed since before new entries were entered into the database, they were first queried.

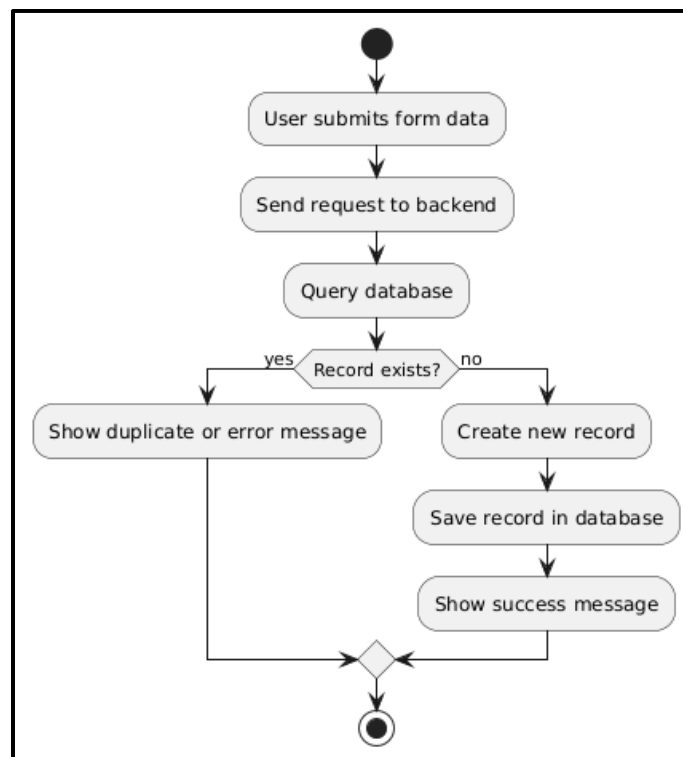


Figure 7 Block Diagram of Database Query and Record Checking Method

3.8 Attendance Percentage Calculation Method

Simple percent computing strategy was employed to show the performance of student attendance. Attendance rate of a subject was determined as the number of the Present records/ the total number of attendance records of a certain subject/ 100. This was rounded to two decimal places and then it was presented in student dashboard and summary pages.

This approach served to offer some clear measure of the performance of students and administrators. Even though it was not complicated, it was good to provide attendance figures in a comprehensible manner.

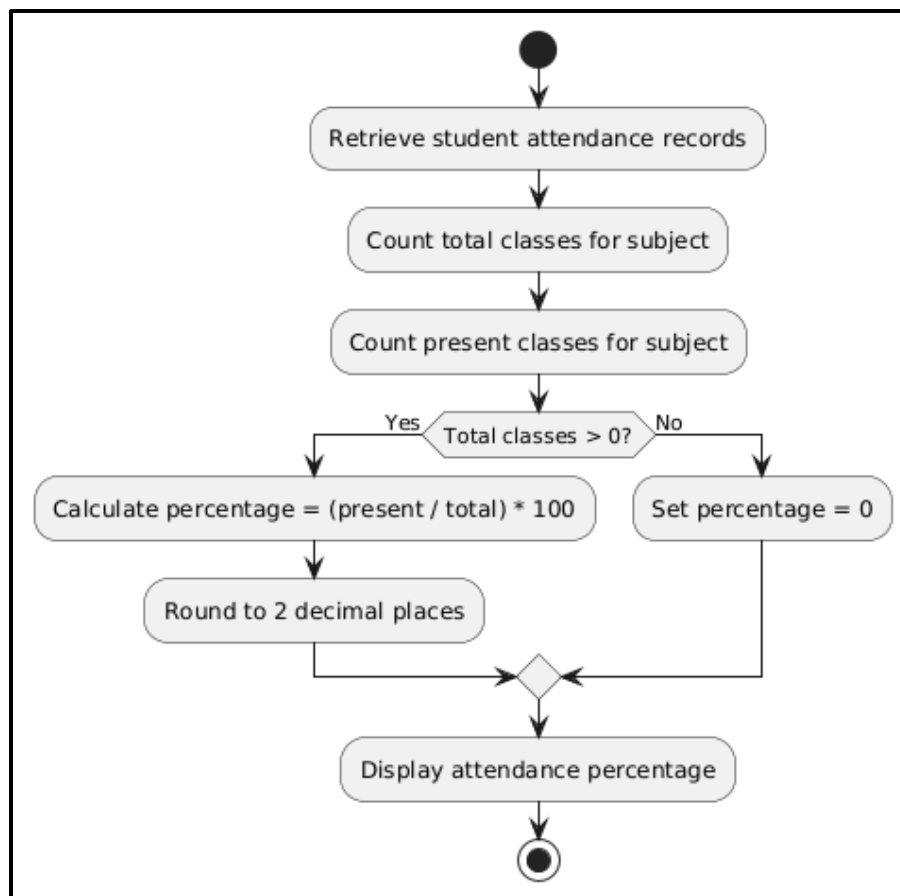


Figure 8 Block Diagram of Attendance Percentage Calculation Method

4.0 Implementation and Critical Analysis

The Attendance Management System was developed with the Flask web framework in Python alongside other supporting applications such as Flask-SQLAlchemy, Flask-Login, Flask-WTF, Flask-Mail and bcrypt. The implementation steps encompassed the design of a secure backend system, database models integration and use of various security controls to resolve common security concerns in a web application. The developer conducted a lot of testing and validation during the development of the development in order to make sure that the implemented functions worked correctly and that the security mechanisms offered provided a good resistance against the potential vulnerabilities. All the security features were developed, tested, and reviewed to ensure that correct solutions were offered to address the security threats identified.

4.1 Secure Password Storage Implementation

An encrypted password store device was adopted based on the hashing algorithm of bcrypt. Whenever implementing it, no password was ever stored in plain text in the database. Rather, they were stored as the result of a conversion of passwords into hashed values with the help of the `bcrypt.hashpw()` function. This approach has made it possible to make sure that although the database may have been compromised, the original user passwords were not readily available.

In the development tests, the hashing process was tested by developing temporary accounts of teachers and students. At the time that the accounts were created, temp passwords were effectively transformed into the hashed values and then store in the database. Then, the process of login testing was carried out using the right and the wrong passwords. The password verification with the help of `checkpw` of the `bcrypt` library verified the correct passwords and denied the wrong ones. The implementation thus made sure that no security of password was compromised anywhere in the system.

```
# Temporary password
temp_password = "Temp@123"
hashed_pw = bcrypt.hashpw(temp_password.encode("utf-8"), bcrypt.gensalt())

new_teacher = User(
    username=form.username.data,
    email=form.email.data,
    password=hashed_pw,
    role="teacher",
    must_change_password=True
)
```

```
temp_password = "Student@123" # temporary password
hashed_pw = bcrypt.hashpw(temp_password.encode("utf-8"), bcrypt.gensalt())

new_student = User(
    username=form.username.data,
    email=form.email.data,
    password=hashed_pw,
    role="student",
    must_change_password=True
)
```

```
if bcrypt.checkpw(form.password.data.encode("utf-8"), user.password):
```

Figure 9 Secure Password Storage Code

4.2 Login Attempt Limitation Implementation

A mechanism of login attempt limit was introduced as a measure against the attempts of the brute-force attack. The failed attempt field in the User database model in the system implementation captures the number of failed attempts made by the user to log in. A failed attempt is counted each time an incorrect password has been entered.

In the process of experimental testing, there were deliberate wrong attempts to log in to the system to test the system reaction. Five unsuccessful attempts led to the automatic locking of the account, in which a value was placed in the lock_until field. The system then blocked any further attempts at logging in until the time period of locking out expired. The outcomes of these tests proved that the account lock system was operating properly and restricting the number of aids to the repeated attempts of the unauthorized logins.

```
else:
    user.failed_attempts += 1

    # Lock after 5 attempts
    if user.failed_attempts >= 5:
        user.lock_until = datetime.utcnow() + timedelta(minutes=5)
        remaining_seconds = 5 * 60
        flash("Too many failed attempts. Account locked for 5 minutes or verify via email.")
        session['locked_user'] = user.username # store username for OTP verification

    else:
        flash("Invalid password")

db.session.commit()
```

```
if user:
    # Check if account locked
    if user.lock_until and datetime.utcnow() < user.lock_until:
        remaining_seconds = int((user.lock_until - datetime.utcnow()).total_seconds())
        flash(f"Account locked. Try again in {remaining_seconds} seconds.")
```

Figure 10 Login Attempt Limitation Code

4.3 CAPTCHA Verification Implementation

The use of CAPTCHA verification helped to avoid automated logins attacks carried out by bots with ill intentions. CAPTCHA mechanism is activated once three attempts to log in are failed and this gives an extra step before more attempts can be entered into the system.

Wrong logins were initially tested to turn on the CAPTCHA. After the activation, the system had to generate a random CAPTCHA image via the `generate captcha()` function. The anticipated CAPTCHA was stored in session and the user input was matched. Several tests were carried out to ensure that people were accurate when doing the validation process. Wrong entries on CAPTCHA blocked the authentication process effectively and the right entries continued with the authentication process. The implementation thus proved that the CAPTCHA mechanism is efficient to distinguish human subjects and automated attack scripts.

```
if user.failed_attempts >= 3:
    if form.captcha.data != session.get("captcha_text"):
        flash("Invalid CAPTCHA")
    return render_template("login.html", form=form, remaining_seconds=remaining_seconds, failed_attempts=user.failed_attempts)
```

```
@app.route('/captcha')
def captcha():
    text, image_data = generate_captcha()

    session['captcha_text'] = text

    return send_file(
        io.BytesIO(image_data.read()),
        mimetype='image/png'
    )
```

```
def generate_captcha(length=5):
    # Characters for captcha
    chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    text = "".join(random.choice(chars) for _ in range(length))
```

Figure 11 CAPTCHA Verification Code

4.4 OTP-Based Account Recovery Implementation

An account recovery process was introduced based on OTP mechanism to offer a secure process in unlocking user accounts that have been temporarily locked following a series of unsuccessful logins. When an account is lock, the user is provided with the opportunity to prove himself or herself with the help of an OTP which is sent to his or her registered email address.

Trial runs of implementation were done with delays as people deliberately locked accounts by typing in wrong passwords. The verification codes were then requested to test the OTP system. The `random.randint()` was used to produce a random six-digit OTP, which was emailed with the help of Flask-Mail. The OTP and its time of expiry were stored in the user session. In the course of the testing, the right OTP access was realized to unlock the account and redefine the counter of the unsuccessful attempts to log in. Further experiments established that OTP values that had expired were rejected and that the resend count was restricted to three. These findings showed that the OTP system offered a secure and controlled account recovery system.

```
otp = str(random.randint(100000, 999999))
otp_expiration = datetime.utcnow() + timedelta(minutes=1)

session['otp'] = otp
session['otp_expiration'] = otp_expiration.timestamp()
session['locked_user'] = user.username
session['resend_count'] = 0
```

```
msg = Message(
    subject="Your Login OTP",
    recipients=[user.email]
)

msg.body = f"Your OTP is {otp}. It expires in 1 minute."
mail.send(msg)
```

```
if entered_otp == otp:
    flash("OTP verified! You can now login.")

    session.pop("otp")
    session.pop("otp_expiration")
    session.pop("resend_count", None)

    username = session.get("locked_user")
    user = User.query.filter_by(username=username).first()

    if user:
        user.failed_attempts = 0
        user.lock_until = None
        db.session.commit()

    return redirect(url_for("login"))
```

```
resend_count = session.get("resend_count", 0)
if resend_count >= 3:
    flash("Maximum OTP resend attempts reached.")
    return redirect(url_for("login"))
```

Figure 12 OTP-Based Account Recovery Code

4.5 Role-Based Access Control Implementation

Role-Based Access Control (RBAC) was used to control access to the system according to the role of a particular user. The system has three user roles including administrator, teacher, and student.

All of the roles were assigned limited access so that only functionalities needed by the user are accessible to them.

Several login scenarios were tested in the process of implementation testing with the use of accounts of various roles. There were efforts to view restricted pages with illegal roles. As an illustration, student accounts would seek to use administrator capabilities of user management and subject creation. Those endeavors were blocked and the users were redirected to the login page. The findings validated the correct implementation of the RBAC that deterred access restrictions and unauthorized access to sensitive system features.

```
@app.route('/admin_dashboard')
@login_required
def admin_dashboard():
    if current_user.role != "admin":
        return redirect(url_for("login"))
```

```
@app.route('/teacher_dashboard')
@login_required
def teacher_dashboard():
    if current_user.role != "teacher":
        return redirect(url_for("login"))
```

```
@app.route('/student_dashboard')
@login_required
def student_dashboard():
    if current_user.role != "student":
        return redirect(url_for("login"))
```

Figure 13 Role-Based Access Control Code

4.6 Database Validation and Data Integrity Implementation

The validation mechanisms in the database were used to make sure that duplicate or invalid data were not inserted in the system. The database was checked to determine the existence of similar records before new records i.e. user, subject or enrollment were created.

When they were testing it, they did it more than once to test the validation logic of the system. As an example, the efforts were made to make user accounts based on current usernames and email addresses. These duplicates were properly identified by the system and the right error messages were shown, without having to add the records to the database as the duplicates. Likewise, duplicate attendance records were avoided with the help of checking an availability of an attendance record with the same student, subject, and date. Such tests proved that the deployed validation mechanisms were effective in the preservation of integrity in databases.

```
existing_username = User.query.filter_by(username=form.username.data).first()
if existing_username:
    flash("Username already exists.")
    return redirect(url_for("create_teacher"))

# ☒ Check duplicate email
existing_email = User.query.filter_by(email=form.email.data).first()
if existing_email:
    flash("Email already exists.")
    return redirect(url_for("create_teacher"))
```

```
existing_username = User.query.filter_by(username=form.username.data).first()
if existing_username:
    flash("Username already exists")
    return redirect(url_for("create_student"))

existing_email = User.query.filter_by(email=form.email.data).first()
if existing_email:
    flash("Email already exists")
    return redirect(url_for("create_student"))
```

```
existing = Subject.query.filter_by(name=form.name.data).first()
if existing:
    flash("Subject already exists")
    return redirect(url_for("create_subject"))
```

```
existing = TeacherStudentEnrollment.query.filter_by(
    teacher_id=teacher_id,
    student_id=student_id,
    subject_id=subject_id
).first()

if existing:
    flash("Student already enrolled for this subject under this teacher.")
```



```
existing = Attendance.query.filter_by(
    user_id=student_id,
    subject_id=subject_id,
    date=today
).first()

if existing:
    flash("Attendance already marked today.")
```

Figure 14 Database Validation and Data Integrity Code

4.7 Attendance Calculation and Data Processing Implementation

A calculation of attendance technique was adopted to give the attendance figures of students and administrators. The system uses the database to retrieve the attendance records and then divides the records of the number of Present records by the total number of present classes recorded under a given subject to give the percentage of attendance.

In the implementation testing, attendance was manually entered and tested by various test conditions. The system was able to compute the percentages of the attendance correctly and show them on the student dashboard and the administrative summary pages. Also the edge cases were tested like zero attendance records in subjects. The system was right in such cases in that a zero percentage value was used to avoid division errors. These tests validated the fact that the attendance calculation logic was working correctly and reliably.

```
for subject in subjects:
    total_classes = Attendance.query.filter_by(
        user_id=current_user.id,
        subject_id=subject.id
    ).count()

    present_classes = Attendance.query.filter_by(
        user_id=current_user.id,
        subject_id=subject.id,
        status="Present"
    ).count()

    percentage = round((present_classes / total_classes) * 100, 2) if total_classes > 0 else 0
    subject_percentages[subject.name] = percentage

return render_template(
    "student_dashboard.html",
    subjects=subjects,
    subject_percentages=subject_percentages
```

```
total_records = Attendance.query.filter_by(subject_id=subject.id).count()

total_present = Attendance.query.filter_by(
    subject_id=subject.id,
    status="Present"
).count()

total_absent = Attendance.query.filter_by(
    subject_id=subject.id,
    status="Absent"
).count()

percentage = 0
if total_records > 0:
    percentage = round((total_present / total_records) * 100, 2)
```

Figure 15 Attendance Calculation and Data Processing Code

4.8 Overall System Evaluation

The introduction of the Attendance Management System proved that all the suggested security measures and functional elements were introduced without significant mistakes. A lot of testing on each module was also done to ensure that authentication, authorization, data validation and account recovery mechanisms worked properly. Security measures, including bcrypt password hashing, restricted number of login attempts, CAPTCHA authentication, unlocking the account with OTP, roles-based access control, and database validation were deployed and tested in a variety of situations.

The outcomes of such studies validated that the system serves well in the mitigation of some of the fundamental web security threats such as brute force attacks, automated logins, unauthorized access to the system, and poor use of passwords. Thus, the adopted system is a stable and safe solution to the issue of attendance organization within a school setting and the security of the user data and system resources.

5.0 Conclusion

This report on implementation demonstrates that a secure Web-Based Attendance Management System was developed using the Flask web framework and a number of support technologies. The system was aimed at automating the student attendance management process with the correct security measures being taken to secure the user data as well as the functionality of the system. Several system features were put in place such as user authentication, role-based access control, attendance management, and administrative management. With the combination of all these elements, a functional and organized system of keeping track of attendance in a learning institution was obtained.

It used a number of security measures to take care of the vulnerabilities associated with web applications. The user credentials were protected by password hashing through bcrypt and limited number of attempts to log in and use CAPTCHA was implemented to minimize the chances of brute force and automated attacks. Moreover, a two-factor authentication system that relied on an OTP was put in place to offer a secure means of account recovery in situations where accounts had been locked because of unsuccessful attempts to log into them. The role based access was also imposed so that system functions were only accessible to authorized system users. The database validation checks were implemented so that no duplication of records is recorded in the system and also to check the integrity of data in the system.

In the process of implementation, several trials and functional tests were performed whereby, the system was tested to make sure it worked well and that the security systems were functioning as expected. The findings revealed that the deployed features effectively dealt with the determined security threats without affecting the usability and performance of the system. All in all, the created system is a dependable and secure way of attending control within an educational setting and reflects the effectiveness of web development and cybersecurity concepts implementation in practice.

6.0 Reference

- OWASP. (2017). *Authentication · OWASP Cheat Sheet Series*. Owasp.org. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Guide, D. (2025). *Implement Digital Identity - OWASP Developer Guide*. Owasp.org. <https://devguide.owasp.org/en/04-design/02-web-app-checklist/06-digital-identity/>
- Huang, S. (2022). Exploratory Analysis of Password and Login Security Methods. *Exploratory Analysis of P Y Analysis of Password and Login Security Methods D and Login Security Methods*, 18. https://digitalcommons.odu.edu/cgi/viewcontent.cgi?params=/context/covacci-undergraduateresearch/article/1018/&path_info=Sofia_Huang_Exploratory_Analysis_of_Password_and_Login_Security_Methods.pdf
- JUN KIM, S., & MUN LEE, B. (2026). *IEEE xplore full-text PDF*: Ieee.org. <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10759630>
- Goud Kothinti, K. (2025). Mitigating one-time passcode (OTP) fraud: Strengthening authentication against emerging threats. *World Journal of Advanced Research and Reviews*, 26(1), 1368–1378. <https://doi.org/10.30574/wjarr.2025.26.1.1181>
- Nawrocki, M., & Kołodziej, J. (2024). Vulnerabilities of Web Applications: Good Practices and New Trends. *Applied Cybersecurity & Internet Governance*. <https://doi.org/10.60097/acig/199521>
- Paul, A., Sharma, V., & Olukoya, O. (2024). SQL injection attack: Detection, prioritization & prevention. *Journal of Information Security and Applications*, 85, 103871–103871. <https://doi.org/10.1016/j.jisa.2024.103871>
- Ablahd, A., & Dawwod, S. (2020). Using flask for SQLIA detection and protection. *Tikrit Journal of Engineering Sciences*, 27(2), 1–14. <https://doi.org/10.25130/tjes.27.2.01>